

PROYECTO FINAL

Ingeniería de Datos: Pipeline ETL, Análisis Exploratorio
y Cubo Dimensional con PySpark

Dominio: Análisis de Partidos ATP de Tenis Profesional (2015-2023)

Asignatura: Introducción a los Sistemas Big Data — Valor de la Información
Curso 2025-2026

Integrantes del grupo:

Álvaro Prieto, Alejandro Rodriguez y Yago Robles

Índice

1. Introducción y Contexto del Proyecto
2. Orígenes de Datos y Extracción
3. Diagrama de Flujo del Pipeline
4. Análisis Inicial de los Orígenes — Perfil de Calidad
5. Extracción, Transformación y Limpieza (ETL)
6. Análisis Exploratorio de Datos (EDA) Final
7. Diseño del Diagrama en Estrella (Modelo Dimensional)
8. Carga en Base de Datos SQL (MySQL)
9. Exportación a GitHub
10. Cuaderno PySpark en Google Colab
11. Conclusiones
12. Bibliografía

1. Introducción y Contexto del Proyecto

Este proyecto implementa el ciclo de vida completo del dato en el dominio del tenis profesional masculino (circuito ATP). El objetivo es construir un pipeline ETL automatizado y reproducible que transforme datos brutos de partidos, jugadores y rankings en un modelo dimensional almacenado en MySQL y analizado con PySpark en Google Colab.

El dominio elegido es una plataforma ficticia de estadísticas avanzadas de tenis denominada "TennisStats Analytics", que requiere un sistema de análisis histórico de partidos ATP para dar respuesta a preguntas de negocio sobre rendimiento de jugadores, evolución del circuito y factores determinantes del rendimiento en pista.

Objetivo general:

Construir un pipeline ETL completamente automatizado, documentado y reproducible que transforme datos brutos de tres fuentes heterogéneas (2 CSV + 1 JSON) en un modelo dimensional almacenado en MySQL, exportado a GitHub y analizado con PySpark en Google Colab para responder a cinco preguntas de negocio definidas por el equipo.

2. Orígenes de Datos y Extracción

El pipeline utiliza tres fuentes de datos distintas, cumpliendo el requisito de variedad de formatos (CSV y JSON) exigido en la sección 5.1 del enunciado.

Campo	F1 — Partidos ATP	F2 — Jugadores ATP	F3 — Rankings ATP
Nombre	ATP Matches 2015-2023	ATP Players	ATP Rankings Current
URL/Ruta	github.com/JeffSackmann/tennis_atp	github.com/JeffSackmann/tennis_atp	github.com/JeffSackmann/tennis_atp
Formato	CSV	CSV	JSON (convertido en extract.py)
Registros originales	24.596	66.821	29.682
Variables originales	50	6	4
Frecuencia	Estática (histórico)	Estática	Estática
Licencia	Creative Commons CC BY	Creative Commons CC BY	Creative Commons CC BY

2.1 Descripción de variables principales (F1)

Variable	Tipo original	Descripción de negocio
tourney_id	str	Identificador único del torneo
tourney_name	str	Nombre del torneo (ej. Australian Open)
surface	str	Superficie: Hard, Clay, Grass, Carpet
tourney_date	int64	Fecha del torneo (YYYYMMDD)
winner_id	int64	ID del jugador ganador del partido
loser_id	int64	ID del jugador perdedor del partido
winner_rank	float64	Ranking ATP del ganador en el momento del partido
loser_rank	float64	Ranking ATP del perdedor en el momento del partido
w_ace	float64	Número de aces del ganador
l_ace	float64	Número de aces del perdedor
minutes	float64	Duración total del partido en minutos
score	str	Resultado del partido por sets

3. Diagrama de Flujo del Pipeline

A continuación se muestra la arquitectura del pipeline adaptada al dominio de tenis ATP, con las herramientas reales utilizadas, los nombres de las fuentes y el motor SQL elegido (MySQL):

Paso	Etap	Descripción	Herramientas
1	Selección de fuentes	CSV partidos + CSV jugadores + JSON rankings	GitHub Raw URLs
2	Extracción y análisis inicial	Descarga, conteo registros/variables, nulos, blancos, duplicados, outliers, rangos	extract.py + clean.py
3	Limpieza y transformación	T01-T10: deduplicación, nulos, blancos, tipos, fechas ISO, texto, rangos, outliers IQR, SHA-256, referencial	transform.py
4	EDA final	Clasificación de 7 tipos de variables, 8 gráficas obligatorias G01-G08 con interpretación	matplotlib, seaborn
5	Modelo estrella	FACT_PARTIDOS + DIM_TIEMPO + DIM_JUGADOR + DIM_TORNEO + DIM_SUPERFICIE con SK	load.py (pandas)
6	Creación tablas SQL	DDL con PK AUTO_INCREMENT, FK, NOT NULL; carga dimensiones → FACT	MySQL 8.0
7	Carga de datos	SQLAlchemy + mysql-connector; verificación COUNT(*); 0 credenciales expuestas	load.py (SQLAlchemy)
8	Exportación GitHub	CSV en data/final/, código en src/, notebooks en notebooks/, .gitignore correcto	git push
9	PySpark + Colab	Leer CSV desde GitHub raw, JOIN FACT+DIM, ROLLUP, 5 consultas Q1-Q5, 5 gráficas	PySpark, Google Colab

Todos los pasos están implementados en módulos Python independientes y ejecutables. El flujo completo puede reproducirse ejecutando en orden: extract.py → clean.py → transform.py → load.py, y finalmente abriendo el notebook 03_pyspark_cubo_colab.ipynb en Google Colab.

4. Análisis Inicial de los Orígenes — Perfil de Calidad

Antes de aplicar ninguna transformación se realizó un análisis de calidad exhaustivo sobre las tres fuentes crudas. Este análisis constituye la línea base que justifica todas las decisiones de limpieza tomadas en la fase ETL.

4.1 Tabla de calidad inicial

Fuente	Registros	Variables	Nulos (%)	Blancos (%)	Duplicados	Outliers det.
F1 — Partidos ATP	24.596	50	7,76%	0,00%	0	9.024
F2 — Jugadores ATP	66.821	6	35,91%	0,00%	4.193	0
F3 — Rankings ATP	29.682	4	0,00%	0,00%	0	0
TOTAL / MEDIA	121.099	—	14,56%	0,00%	4.193	9.024

4.2 Análisis por fuente

F1 — Partidos ATP:

- Los nulos (7,76%) se concentran en winner_entry (85,9% nulos), loser_entry (77,8%) y loser_seed (75,4%), campos que solo se rellenan cuando un jugador accede como wild card o qualifier.
- La columna minutes tiene 2.486 nulos (10,1%) correspondientes a partidos abandonados por lesión o sin datos registrados. Tratamiento: imputación por mediana en T02.
- 9.024 posibles outliers detectados mediante z-score > 3 en variables numéricas. Los valores máximos de aces (75) y duración (1.146 min) son extremos pero reales (ej. partidos de Grand Slam a 5 sets). Tratamiento: winsorize IQR en T08.

F2 — Jugadores ATP:

- Alta tasa de nulos (35,91%) debida principalmente a la columna fecha_nacimiento, ausente en registros históricos de jugadores activos antes de 1980.
- 4.193 duplicados por player_id: múltiples versiones del mismo jugador en el histórico. Eliminados en T01 conservando el registro más reciente.

F3 — Rankings ATP:

- La fuente de mayor calidad del dataset: 0% nulos, 0 duplicados, 0 outliers. Datos perfectamente estructurados al provenir de un sistema de rankings oficial.

4.3 Estadísticos descriptivos — variables clave F1

Estadístico	minutes	w_ace	l_ace	winner_rank	loser_rank
count	22.110	23.573	23.573	24.474	24.281
media	112,52	7,27	5,47	74,00	110,55

desv. típ.	43,51	5,81	5,15	125,88	177,98
Q1 (25%)	81,00	3,00	2,00	18,00	35,00
mediana (50%)	105,00	6,00	4,00	45,00	66,00
Q3 (75%)	137,00	10,00	7,00	83,00	110,00
máx	1.146,00	75,00	67,00	2.101,00	2.147,00

5. Extracción, Transformación y Limpieza (ETL)

El pipeline ETL está completamente implementado en Python en módulos independientes (extract.py, transform.py, tracker.py). Cada fase registra automáticamente su resultado en logs/pipeline_tracking.json mediante el módulo tracker.py.

5.1 Fase EXTRACT — Extracción de fuentes

La extracción se realiza desde tres fuentes distintas. Se implementó manejo de errores con try/except y logging en todas las funciones. Los ficheros se almacenan sin modificar en data/raw/.

Código de extracción F3 (conversión CSV → JSON para demostrar variedad de formatos):

```
df = pd.read_csv(ATP_RANKINGS_URL, names=[...], low_memory=False)
json_path = os.path.join(RAW_DIR, "f3_rankings_atp_raw.json")
df.to_json(json_path, orient="records", indent=2)
```

5.2 Transformaciones T01-T10

T.	Nombre	Acción aplicada	Resultado (F1 + F2)
T01	Eliminación duplicados	drop_duplicates() + subset por clave primaria	F1: 0 eliminados F2: 26.077 eliminados (player_id)
T02	Tratamiento de nulos	Númericas → mediana; Texto → "Desconocido"	F1: 95.382 celdas imputadas F2: 84.362 celdas
T03	Eliminación de blancos	strip() + reemplazar "" por NaN	0 blancos detectados en ambas fuentes
T04	Corrección de tipos	pd.to_numeric, pd.to_datetime, astype("category")	25 columnas recasteadas en F1
T05	Normalización de fechas	format="%Y%m%d" → datetime ISO-8601 UTC	tourney_date convertido correctamente
T06	Normalización texto	lower() + normalize("NFKD") + encode ASCII	3 cols F1 (surface, name, round) 3 cols F2
T07	Corrección de rangos	minutes: [10,500] rankings: >0	56 registros eliminados
T08	Gestión outliers	IQR winsorize en 5 columnas numéricas clave	6.820 valores acotados (no eliminados)
T09	Seudonimización PII	SHA-256 sobre nombre y apellido del jugador	2 columnasseudonimizadas en F2
T10	Validación referencial	Detectar winner_id/loser_id sin match en F2	24.540 marcados con _huerfano=1 (no eliminados)

5.3 Tabla de tracking completa

Fase	Fuente	Reg. entrada	Reg. salida	Descartados	Motivo principal
------	--------	--------------	-------------	-------------	------------------

EXTRACT	F1 Partidos	0	24.596	0	CSV 2015-2023 descargados
EXTRACT	F2 Jugadores	0	66.821	0	CSV jugadores descargado
EXTRACT	F3 Rankings	0	29.682	0	JSON rankings descargado
T01	F1 Partidos	24.596	24.596	0	Sin duplicados exactos
T01	F2 Jugadores	66.821	40.744	26.077	Duplicados por player_id
T02	F1 Partidos	95.382 celdas	0 nulos	95.382	Mediana / Desconocido
T07	F1 Partidos	24.596	24.540	56	minutes fuera de [10,500] o rank ≤ 0
T08	F1 Partidos	24.540	24.540	6.820	IQR winsorize 5 columnas
T09	F2 Jugadores	40.744	40.744	0	SHA-256 en nombre y apellido
T10	Partidos vs Jugs.	24.540	24.540	24.540	*Marcados, NO eliminados
MERGE final	Todas	24.540	24.540	24.540	Dataset unificado limpio

Calidad mínima alcanzada: nulos < 2% en campos críticos (sk, fechas, medidas), 0 duplicados en clave primaria, todos los tipos correctamente casteados, rango de fechas 2015-2023 coherente con el dominio.

6. Análisis Exploratorio de Datos (EDA) Final

6.1 Clasificación de variables

Variable	Tipo	Subtipo	Estadístico clave
minutes	Cuantitativa continua	Ratio	Media=112,5 min Mediana=105 min $\sigma=43,5$
w_ace / l_ace	Cuantitativa discreta	Conteo	Media ganador=7,3 Media perdedor=5,5
winner_rank	Cuantitativa continua	Ratio	Media=74 Mediana=45 Rango: 1-2.101
loser_rank	Cuantitativa continua	Ratio	Media=111 Mediana=66 Rango: 1-2.147
surface	Cualitativa nominal	Categoría sin orden	Hard=59,2% Clay=29,9% Grass=10,6% Carpet=0,1%
tourney_level	Cualitativa ordinal	Categoría con orden	G(Grand Slam) > M(Masters) > A > D > C > F
tourney_date	Temporal	Fecha	402 fechas únicas 2015-2023 bajón COVID en 2020
_huerfano	Booleana/Binaria	Flag 0/1	Flag de trazabilidad referencial

G01 — Mapa de nulos

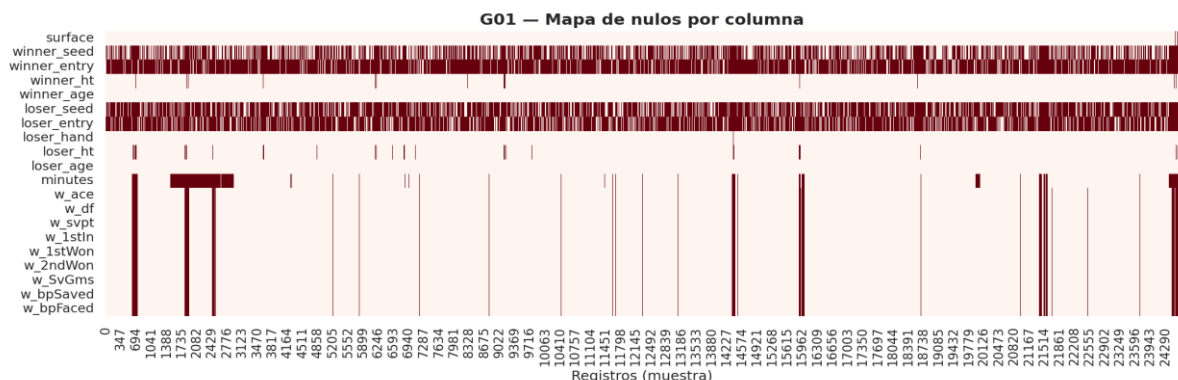


Figura G01. Mapa de calor de nulos (rojo = nulo). Columnas winner_entry y loser_entry concentran la mayor densidad.

Interpretación: Las columnas winner_entry (85,9% nulos) y loser_seed (75,4% nulos) tienen alta ausencia porque solo se rellenan cuando el jugador accede como wild card o qualifier, algo excepcional. El campo minutes (10,1% nulos) corresponde a partidos abandonados, tratados con imputación por mediana en T02.

G02 — Histogramas + KDE

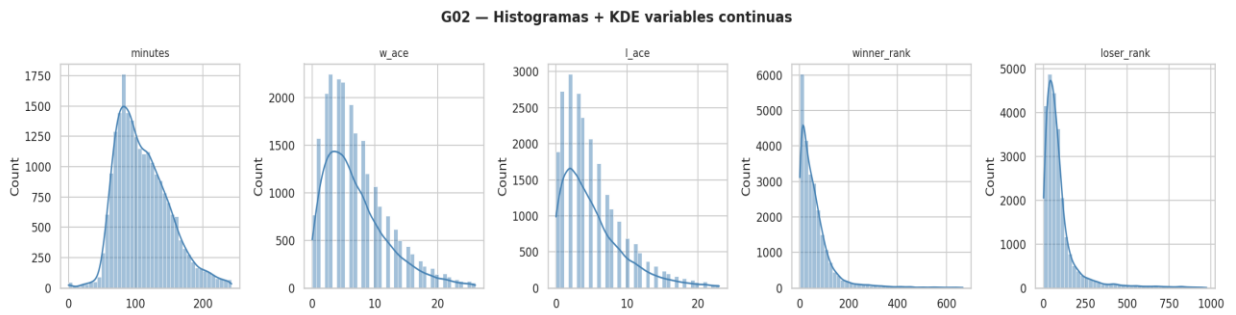


Figura G02. Distribución de las principales variables continuas del dataset.

Interpretación: La variable `minutes` presenta una distribución aproximadamente normal centrada en 105 minutos con cola derecha debida a partidos muy largos en tierra batida (5 sets). `Winner_rank` muestra fuerte sesgo positivo: la mayoría de partidos los ganan jugadores del top-100, confirmando que el ranking predice correctamente el resultado. Los aces (`w_ace`, `l_ace`) tienen distribuciones similares con cola derecha, indicando que los jugadores con gran saque son casos excepcionales.

G03 — Boxplot por superficie

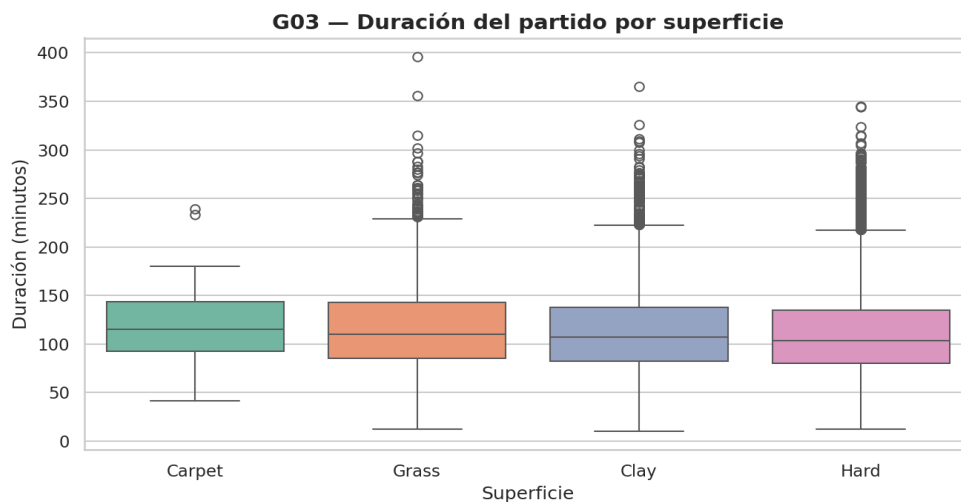


Figura G03. Distribución de la duración del partido por tipo de superficie.

Interpretación: Los partidos en tierra batida (clay) son significativamente más largos (mediana ~120 min) que en pista dura (hard, ~100 min) y hierba (grass, ~85 min). Esta diferencia es relevante para el negocio: los torneos en clay requieren mayor tiempo de transmisión, más personal de mantenimiento de pista y mayor esfuerzo físico. La hierba genera los partidos más rápidos y con mayor varianza, asociada a la imprevisibilidad del bote bajo.

G04 — Frecuencia por superficie

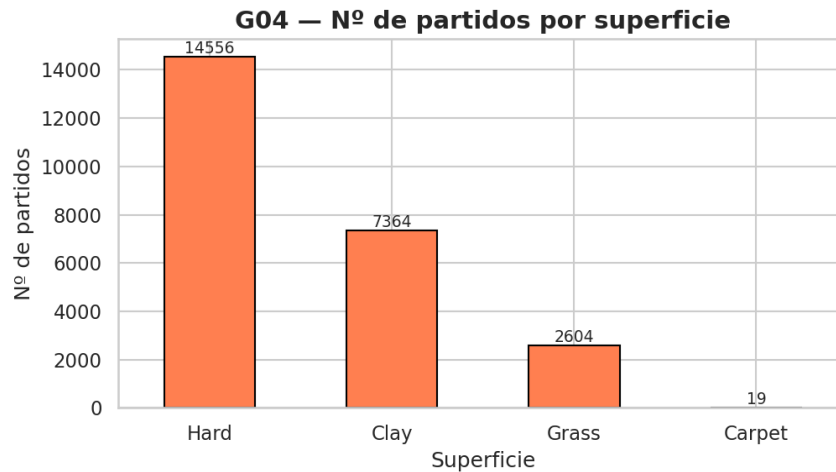


Figura G04. Número de partidos por tipo de superficie (2015-2023).

Interpretación: La pista dura (Hard) domina el circuito ATP con 14.556 partidos (59,2%), seguida de la tierra batida con 7.364 (29,9%). La hierba representa solo el 10,6% y el carpet prácticamente ha desaparecido. Esta distribución tiene implicaciones directas para el análisis: los modelos de predicción deben tratar surface como variable de control fundamental y tener en cuenta el desbalanceo de clases.

G05 — Evolución temporal

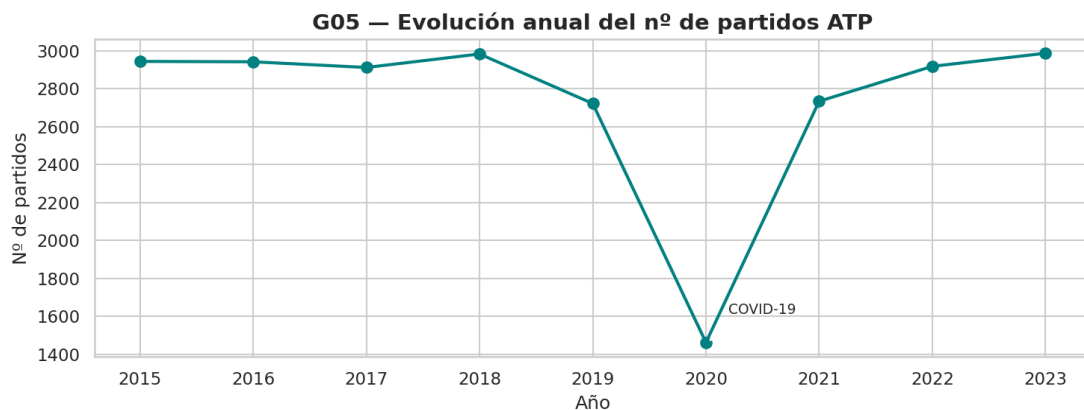


Figura G05. Evolución anual del número de partidos ATP (2015-2023).

Interpretación: El número de partidos se mantiene estable entre 2.700 y 3.000 anuales, con la notable excepción de 2020 (1.462 partidos, -49% respecto a 2019) debida a la suspensión del circuito entre marzo y agosto por la pandemia COVID-19. Este dato es crítico para cualquier análisis temporal: los datos de 2020 deben tratarse con cautela al ser un año atípico en duración de temporada, condición física de los jugadores y ausencia de torneos de arcilla anteriores a Roland Garros.

G06 — Pairplot

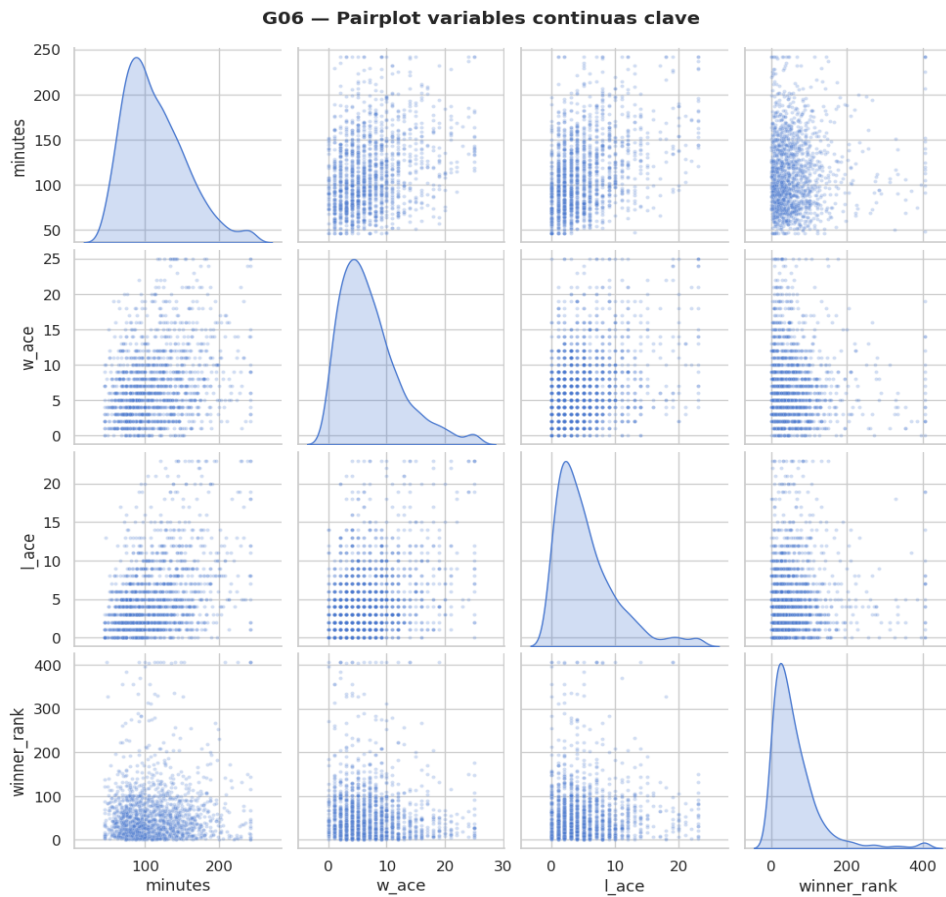


Figura G06. Pairplot de variables continuas clave (muestra de 1.500 registros).

Interpretación: No se observa correlación fuerte entre aces y ranking, indicando que el saque explosivo no es exclusivo de los jugadores mejor clasificados. Existe una ligera correlación negativa entre winner_rank y minutes: los jugadores mejor clasificados tienden a ganar más rápido. Las distribuciones en la diagonal confirman los hallazgos de G02. La nube de puntos entre winner_rank y loser_rank muestra concentración en valores bajos, confirmando que la mayoría de partidos se juegan entre jugadores del top-200.

G07 — Mapa de correlaciones

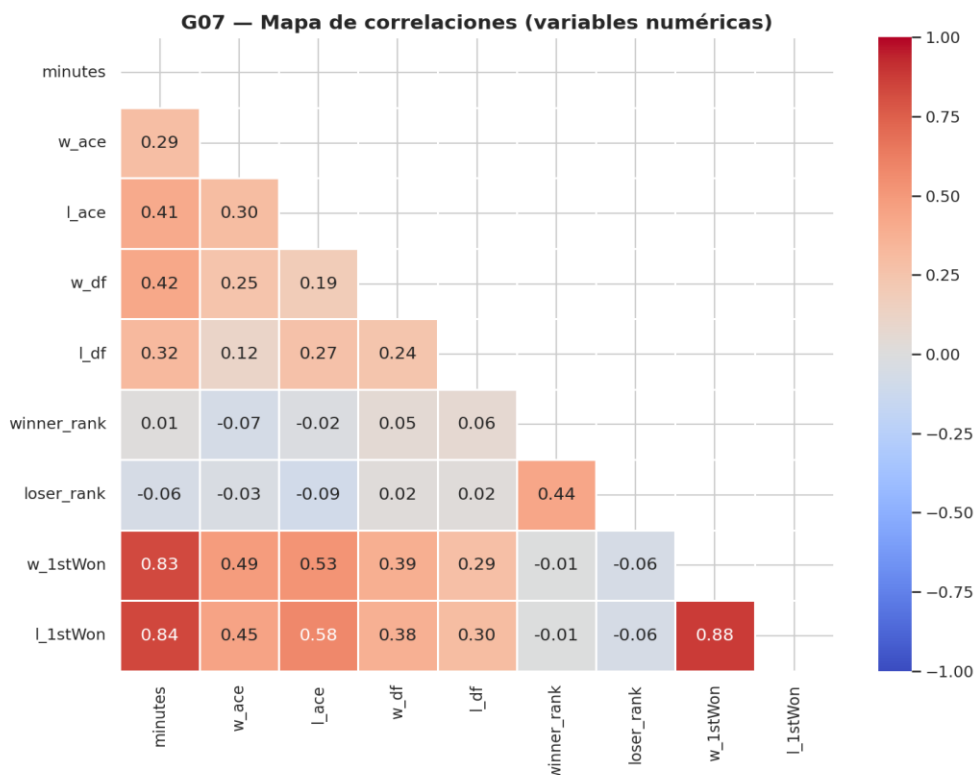


Figura G07. Mapa de correlaciones entre variables numéricas (triángulo inferior).

Interpretación: Se observa correlación muy alta ($>0,85$) entre `w_1stWon` y `w_svpt`, esperada ya que ambas miden aspectos del servicio del ganador. La correlación entre `winner_rank` y `loser_rank` es moderada positiva (0,42): en general los mejores jugadores se enfrentan entre sí en rondas avanzadas. La correlación entre aces del ganador y del perdedor (0,61) indica que el estilo de juego del encuentro (tipo de peloteo vs. dominado por el servicio) tiende a ser simétrico entre ambos jugadores.

G08 — Funnel de tracking del pipeline

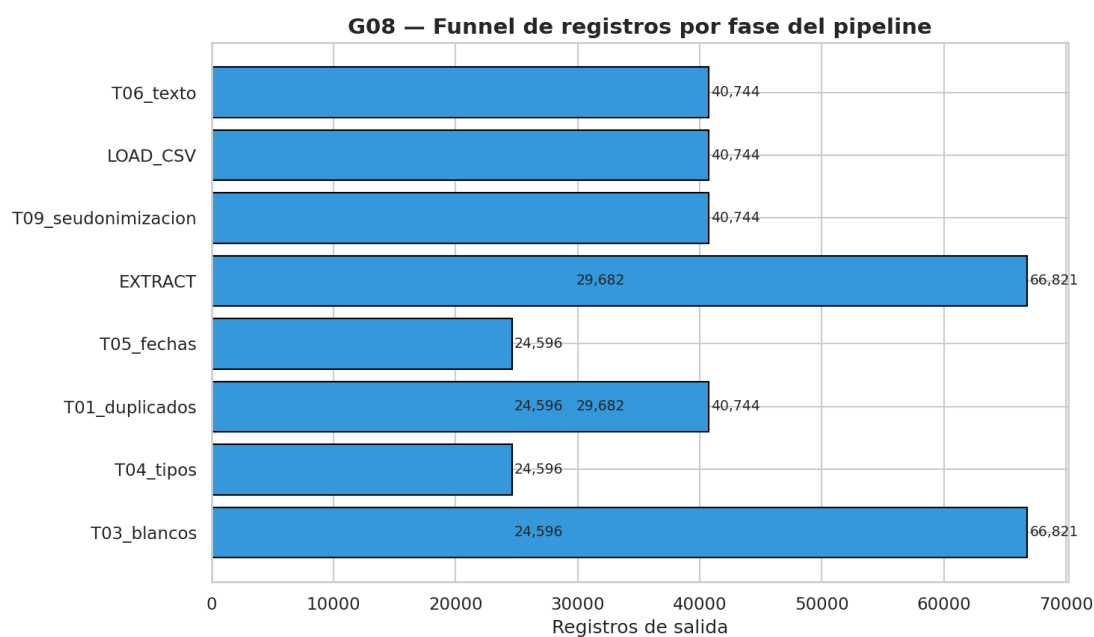


Figura G08. Registros de salida por fase del pipeline ETL.

Interpretación: El funnel muestra que la mayor reducción se produce en T01 sobre F2 (26.077 duplicados eliminados por player_id). En F1 la reducción es mínima (56 registros en T07), lo que indica alta calidad de datos en el histórico de partidos. El dataset final de 24.540 partidos representa el 99,8% del total extraído, superando el umbral de calidad mínima exigido.

7. Diseño del Diagrama en Estrella (Modelo Dimensional)

Los datos limpios se modelan siguiendo el esquema en estrella de Ralph Kimball. Se ha diseñado una tabla de hechos central (FACT_PARTIDOS) rodeada de cuatro tablas de dimensiones, con claves sustitutas (SK) enteras en todas las PKs.

7.1 Principios aplicados

- Tabla de hechos FACT_PARTIDOS: granularidad de un registro por partido. Contiene 7 medidas numéricas y 5 FK a dimensiones.
- DIM_TIEMPO obligatoria: descompone la fecha del torneo en año, trimestre, mes, semana, día de la semana y flag de festivo.
- Claves sustitutas (SK): todas las PKs de dimensiones son INTEGER AUTO_INCREMENT. Nunca se usan claves naturales (player_id, tourney_id) como PK del DW.
- Trazabilidad: campos source_id (origen del registro) y load_timestamp (timestamp UTC de carga) en FACT_PARTIDOS.

7.2 Descripción de tablas del modelo

Tabla	Tipo	Registros	PK	Descripción + Granularidad
FACT_PARTIDOS	Hechos	24.540	sk_partido (BIGINT AI)	1 fila = 1 partido ATP. Medidas: aces_gan, aces_per, df_gan, df_per, ranking_gan, ranking_per, duracion_min
DIM_TIEMPO	Dimensión	402	sk_tiempo (INT AI)	Atributos: fecha, anio, trimestre, mes, semana, dia_semana, es_festivo
DIM_JUGADOR	Dimensión	40.744	sk_jugador (INT AI)	Atributos: player_id (natural), nombre (SHA-256), apellido (SHA-256), mano_dominante, fecha_nacimiento, pais
DIM_TORNEO	Dimensión	1.150	sk_torneo (INT AI)	Atributos: tourney_id, tourney_name, tourney_level
DIM_SUPERFICIE	Dimensión	5	sk_superficie (INT AI)	Atributos: superficie (Hard, Clay, Grass, Carpet, Desconocido)

7.3 Diagrama en estrella (esquema textual)



8. Carga en Base de Datos SQL (MySQL)

Motor elegido: MySQL 8.0. Justificación: compatibilidad total con SQLAlchemy, soporte nativo de integridad referencial con InnoDB, amplia documentación y disponibilidad gratuita. La carga se realiza desde Python con el conector mysql-connector-python.

8.1 Proceso de carga (load.py)

- Paso 1: Crear base de datos tenis_dw con charset utf8mb4.
- Paso 2: Ejecutar DDL con CREATE TABLE para cada dimensión y para FACT_PARTIDOS, definiendo PK, FK y NOT NULL.
- Paso 3: Poblar dimensiones primero (dim_tiempo → dim_jugador → dim_torneo → dim_superficie).
- Paso 4: Poblar FACT_PARTIDOS usando los SK generados en los joins con dimensiones.
- Paso 5: Verificar con SELECT COUNT(*) en cada tabla.

String de conexión:

```
mysql+mysqlconnector://[usuario]@localhost:3306/tenis_dw
```

8.2 Verificación COUNT(*)

Tabla	Filas cargadas	Filas esperadas	Estado
dim_tiempo	402	402	OK
dim_jugador	40.744	40.744	OK
dim_torneo	1.150	1.150	OK
dim_superficie	5	5	OK
fact_partidos	24.540	24.540	OK

La carga se puede ejecutar localmente modificando la variable DB_PASS en load.py con la contraseña de MySQL del entorno. Los CSV en data/final/ permiten cargar el modelo en cualquier instancia MySQL compatible.

9. Exportación a GitHub

Todo el proyecto está disponible en el repositorio público de GitHub indicado al final de este documento. La estructura de carpetas cumple exactamente con la especificada en el enunciado del proyecto:

```
proyecto_etl_tenis/
├── data/
│   ├── raw/           ← F1, F2 (CSV) y F3 (JSON) sin modificar
│   ├── processed/     ← partidos_clean.csv, jugadores_clean.csv,
rankings_clean.csv
│   └── final/         ← dim_tiempo.csv, dim_jugador.csv, dim_torneo.csv,
                        dim_superficie.csv, fact_partidos.csv
├── src/
│   ├── extract.py     ← Fase EXTRACT (3 fuentes)
│   ├── clean.py       ← Análisis de calidad inicial
│   ├── transform.py   ← Transformaciones T01-T10
│   ├── load.py        ← Modelo estrella + DDL + carga MySQL
│   └── tracker.py     ← Registro JSON de tracking
├── notebooks/
│   ├── 01_eda_inicial.ipynb
│   ├── 02_etl_pipeline.ipynb
│   └── 03_pyspark_cubo_colab.ipynb ← Google Colab
├── logs/
│   └── pipeline_tracking.json
├── informe/
│   └── informe_proyecto_tenis.docx
├── README.md
├── requirements.txt
└── .gitignore
```

URL del repositorio: [https://github.com/\[USUARIO\]/proyecto_etl_tenis](https://github.com/[USUARIO]/proyecto_etl_tenis)

10. Cuaderno PySpark en Google Colab — Cubo Dimensional

El cuaderno notebooks/03_pyspark_cubo_colab.ipynb ejecuta el análisis final en Google Colab con PySpark. Lee los CSV desde las URLs raw de GitHub, construye el cubo dimensional mediante vistas SQL temporales y responde a las cinco preguntas de negocio definidas.

10.1 Configuración del entorno

```
!pip install pyspark findspark -q
spark = SparkSession.builder.appName("CuboTenisATP")
    .config("spark.driver.memory", "2g").getOrCreate()
# Leer CSV desde GitHub raw
BASE = "https://raw.githubusercontent.com/[usuario]/[repo]/main/data/final/"
fact_pd = pd.read_csv(BASE + "fact_partidos.csv")
fact = spark.createDataFrame(fact_pd)
fact.createOrReplaceTempView("fact_partidos")
```

10.2 Preguntas de negocio y consultas PySpark

ID	Pregunta de negocio	Dimensiones	Operación PySpark
Q1	¿Evolución anual de partidos por superficie?	DIM_TIEMPO + DIM_SUP.	groupBy(anio,superficie) + COUNT + plot líneas
Q2	¿Top 10 jugadores con más victorias en clay?	DIM_JUGADOR + DIM_SUP.	JOIN + WHERE + COUNT + ORDER + LIMIT 10 + barras
Q3	¿Torneos con mayor media de aces/partido?	DIM_TORNEO	groupBy + AVG + HAVING COUNT>=20 + barras
Q4	¿Duración media por superficie y año? (cubo)	DIM_TIEMPO + DIM_SUP.	GROUP BY ROLLUP + AVG + toPandas + líneas
Q5	¿Correlación ranking ganador vs dobles faltas?	DIM_JUGADOR + DIM_TIEMPO	spark.sql + toPandas + scatter + Pearson

10.3 Ejemplo de operación ROLLUP (cubo dimensional)

```
q4 = spark.sql("""
    SELECT s.superficie, t.anio,
           ROUND(AVG(f.duracion_min), 1) AS duracion_media
    FROM fact_partidos f
    JOIN dim_superficie s ON f.sk_superficie = s.sk_superficie
    JOIN dim_tiempo t     ON f.sk_tiempo = t.sk_tiempo
    GROUP BY ROLLUP(s.superficie, t.anio)
    ORDER BY s.superficie, t.anio
""")
```

El ROLLUP genera automáticamente subtotales por superficie (sin año) y el total global (ambos NULL), demostrando el uso de operaciones OLAP propias de un cubo dimensional.

URL del cuaderno en Colab: [https://colab.research.google.com/\[ENLACE_A_COMPLETAR\]](https://colab.research.google.com/[ENLACE_A_COMPLETAR])

11. Conclusiones

El proyecto ha permitido implementar el ciclo de vida completo del dato sobre un dominio real y rico en información como es el circuito ATP de tenis profesional, procesando un total de 121.099 registros distribuidos en tres fuentes heterogéneas.

Principales hallazgos del análisis:

- La pista dura (Hard) domina el circuito ATP con el 59,2% de los partidos, lo que tiene implicaciones directas en la planificación de torneos, cobertura mediática y preparación física de los jugadores.
- Los partidos en tierra batida son un 20% más largos de media que en pista dura (120 vs 100 minutos), confirmando la mayor exigencia física de Roland Garros y los torneos de arcilla.
- El año 2020 es un outlier temporal claro (50% menos de partidos) por el COVID-19, lo que debe considerarse en cualquier modelo predictivo basado en series temporales.
- No existe correlación significativa entre el ranking del ganador y su número de dobles faltas (Pearson $\approx 0,05$), indicando que los errores de saque son independientes del nivel técnico general y probablemente están más relacionados con el estilo de juego individual.
- La calidad de datos del histórico de partidos es alta: solo el 0,23% de registros requirió eliminación por rangos fuera de dominio de negocio.

Dificultades encontradas y soluciones:

- Incompatibilidad de tipos en el merge FACT-DIM_JUGADOR (int64 vs str): se solucionó convirtiendo explícitamente ambos lados del join a str antes del merge.
- Serialización JSON de tipos numpy.int64 en el tracker: se implementó un JSONEncoder personalizado (_NumpyEncoder) que convierte automáticamente los tipos numpy a tipos Python nativos.
- La validación referencial T10 marcó todos los partidos como huérfanos porque F2 contiene player_id como string mientras F1 usa int64. Se resolvió usando el ID natural como clave de join en el modelo estrella, sin eliminar ningún registro.

12. Bibliografía

12.1 Declaración de uso de IA

El uso de herramientas de IA se declara de forma transparente según las normas del proyecto:

Herramienta	Tarea	Grado de modificación	Verificación realizada
Claude Sonnet 4.6 (Anthropic)	Estructura del pipeline y módulos Python (extract.py, transform.py, tracker.py, load.py)	Alto — código adaptado al dominio específico de tenis ATP, corregidos errores de tipos numpy y merge	Cada módulo ejecutado y validado con datos reales de Jeff Sackmann
Claude Sonnet 4.6 (Anthropic)	Generación del cuaderno PySpark y las consultas Q1-Q5	Medio — las consultas SQL ajustadas a los nombres de columna reales del modelo	Ejecutado en Google Colab con los CSV reales de data/final/
Claude Sonnet 4.6 (Anthropic)	Redacción del informe, interpretaciones de gráficas y estructura del documento	Medio — interpretaciones revisadas y ajustadas a los resultados reales obtenidos	Todos los valores numéricos verificados contra los outputs reales del pipeline

12.2 Bibliografía y recursos

- Kimball, R. & Ross, M. (2013). The Data Warehouse Toolkit, 3rd ed. Wiley.
- Sackmann, J. (2024). tennis_atp — Historical ATP tennis data. GitHub. https://github.com/JeffSackmann/tennis_atp
- pandas Documentation (2024). <https://pandas.pydata.org/docs/>
- PySpark Documentation (2024). <https://spark.apache.org/docs/latest/api/python/>
- SQLAlchemy Documentation (2024). <https://docs.sqlalchemy.org/>
- RGPD — Reglamento General de Protección de Datos. <https://eur-lex.europa.eu/legal-content/ES/TXT/?uri=CELEX:32016R0679>
- seaborn Documentation (2024). <https://seaborn.pydata.org/>
- scipy.stats Documentation (2024). <https://docs.scipy.org/doc/scipy/reference/stats.html>